SIMATS ENGINEERING
ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)*

Dave's Taxonomy: Manipulation (Level 2)

Question Count: 5 **Total Marks: 25**

Code of Conduct

I N.Rama Sai Kiran Reddy(192525044) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:N.Rama Sai

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
The system must handle millions of concurrent users with minimal buffering.
Design a cloud architecture using scalability and caching techniques.
Explain how load balancing and auto-scaling can ensure smooth streaming.
Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
They face challenges in maintaining consistency and managing deployments.
Propose a solution using container orchestration and multi-cloud strategies.
Explain how service discovery and scaling can be handled efficiently.
Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
The organization needs a robust backup and disaster recovery strategy.
Design a solution using cloud-native backup, replication, and versioning.
Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.

Some applications must remain on-premises due to compliance requirements.
Design a hybrid architecture that ensures secure communication between environments.
Explain how identity management and encryption should be implemented.
Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down.
Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured.
Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1. A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

Design a cloud-based big data architecture using distributed storage and processing frameworks.

Explain how batch processing and stream processing can be integrated.

Discuss how data lakes and data warehouses can be used together.

Recommend techniques to ensure scalability and cost efficiency.

2. A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

Design a secure cloud architecture that protects sensitive healthcare data.

Explain how encryption, identity and access management (IAM), and key management should be implemented.

Discuss auditing, logging, and compliance monitoring strategies.

Recommend methods to prevent unauthorized access and data breaches.

3. An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

Design a serverless cloud architecture for the application.

Explain how event-driven computing can improve scalability.

Discuss the role of managed databases, API gateways, and message queues.

Evaluate the advantages and limitations of serverless computing for this workload.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.

Explain how data quality, metadata management, and governance should be maintained.

Discuss the role of workflow orchestration and scheduling tools.

Recommend strategies for optimizing query performance and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

Design a cloud-based machine learning and analytics architecture.

Explain how big data technologies can support model training and inference.

Discuss the use of cloud-native AI/ML services for model deployment and monitoring.

Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

Design a cloud migration strategy suitable for the organization.

Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.

Discuss methods to validate application performance after migration.

Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

Design a cloud-based architecture for IoT data collection and analytics.

Explain how message brokers, stream processing, and cloud storage are integrated.

Discuss methods for handling high-volume, high-velocity sensor data.

Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

Design a cloud-based business intelligence architecture.

Explain how data visualization tools integrate with cloud data warehouses.

Discuss techniques for optimizing dashboard performance with large datasets.

Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

Design a cloud monitoring and performance optimization solution.

Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.

Discuss strategies for resource optimization and capacity planning.

Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

Design a cloud-based data lifecycle management strategy.

Explain how data classification, retention policies, and archival storage should be implemented.

Discuss the importance of governance, auditing, and regulatory compliance in big data environments.

Recommend techniques for balancing data accessibility, security, and cost optimization.

SET 1 – Cloud Computing, Virtualization and Deployment (CO2)

1. Video Streaming Platform Migration

Cloud architecture using scalability and caching

- **Ingest and transcoding:** Raw video is uploaded to object storage (e.g., Amazon S3 / Azure Blob). A serverless or container-based transcoding pipeline converts it into multiple bitrates and formats (HLS/DASH) for adaptive streaming.
- **Origin and storage tier:** Encoded segments are stored in highly durable, replicated object storage; metadata (titles, users, watch history) lives in a managed NoSQL/SQL database.
- **Caching tier:** A global CDN (CloudFront, Azure CDN, Cloud CDN) caches video segments at edge locations close to viewers. An in-memory cache (Redis/Memcached) caches catalogue, session and recommendation data.
- **Application tier:** Stateless microservices (authentication, catalogue, playback, billing) run on VMs or Kubernetes behind a load balancer so any instance can serve any request.
- **Scalability:** Horizontal scaling across multiple Availability Zones; read replicas and database sharding handle read-heavy traffic.

Load balancing and auto-scaling for smooth streaming

- **Load balancing:** Global DNS/Anycast routing sends users to the nearest healthy region; Layer-7 load balancers distribute API traffic using round-robin or least-connections, with health checks removing failed nodes.
- **Auto-scaling:** Scaling policies add or remove instances/pods based on CPU, memory, request rate or concurrent-stream metrics. Scheduled scaling prepares for known peaks (new releases, live events); predictive scaling uses historical patterns.
- **Result:** Because the CDN serves most traffic from the edge and the application tier scales out automatically, buffering and start-up delay stay low even with millions of concurrent users.

Cost optimization strategies

- Use a high CDN cache-hit ratio to cut origin and data-transfer costs; negotiate committed-use CDN pricing.
- **Mix pricing models:** Reserved/Savings Plans for baseline load, On-Demand for normal spikes, Spot instances for fault-tolerant transcoding jobs.
- **Storage tiering:** Move rarely watched content to infrequent-access or archive tiers using lifecycle policies.
- **Efficient encoding:** Per-title encoding and modern codecs (H.265/AV1) reduce bandwidth per viewer.
- **Right-sizing and monitoring:** Use cost-monitoring tools, budgets and alerts; shut down idle resources and tag resources for cost allocation.

2. Fintech Containers Across Multiple Cloud Providers

Container orchestration and multi-cloud solution

- Package every service as a Docker image stored in a private, signed container registry, so the same artefact runs on every cloud.
- Use Kubernetes (EKS, AKS, GKE or a multi-cloud platform such as Anthos/Rancher/OpenShift) as the common orchestration layer across providers.
- Manage infrastructure with Infrastructure-as-Code (Terraform) and deploy with GitOps (Argo CD / Flux) so the desired state is defined in Git and applied consistently to every cluster.
- Use Helm charts or Kustomize for environment-specific configuration, and a CI/CD pipeline with automated security scanning and policy checks (OPA/Gatekeeper).
- Centralise identity, secrets (Vault) and observability (Prometheus, Grafana, ELK) so operations look the same on all clouds.

Service discovery and scaling

- **Service discovery:** Kubernetes DNS and Services give each microservice a stable name; a service mesh (Istio/Linkerd/Consul) adds cross-cluster discovery, mTLS, retries and traffic routing.
- **Scaling:** Horizontal Pod Autoscaler scales pods on CPU/memory/custom metrics; Cluster Autoscaler adds or removes nodes; KEDA scales on event/queue depth.
- **Traffic management:** A global load balancer or DNS-based routing distributes requests between clouds and shifts traffic during failures.

Benefits and risks of multi-cloud

- **Benefits:** Avoids vendor lock-in, improves resilience if one provider fails, allows best-of-breed services, supports data-residency and regulatory needs, and strengthens negotiating power.
- **Risks:** Higher complexity and skill requirements, inconsistent security models and tooling, data-transfer (egress) charges, latency between clouds, harder cost tracking, and a larger attack surface.
- **Mitigation:** Standardise on Kubernetes and IaC, enforce uniform policies, use unified monitoring and FinOps tools, and encrypt all inter-cloud traffic.

3. E-Learning Platform Data Loss and Disaster Recovery

Solution using cloud-native backup, replication and versioning

- **Versioning:** Enable object versioning on storage buckets and database point-in-time recovery, so an accidental delete only creates a delete marker and earlier versions can be restored.
- **Deletion protection:** Use MFA-delete, object lock/WORM, soft delete and least-privilege IAM so that only authorised roles can permanently delete data.
- **Backups:** Automated, scheduled snapshots of databases and volumes through AWS Backup / Azure Backup / Cloud Backup, with retention policies (daily, weekly, monthly) and encryption.

- **Replication:** Cross-zone and cross-region replication of storage and databases; copy backups to a separate account/region to protect against account compromise or regional failure.
- **Testing:** Regularly restore backups in a test environment to verify integrity.

How RTO and RPO are achieved

- **RPO (maximum acceptable data loss):** Achieved through backup frequency, continuous replication and transaction-log/point-in-time recovery. Near-zero RPO needs synchronous or continuous asynchronous replication.
- **RTO (maximum acceptable downtime):** Achieved through the DR pattern chosen – Backup & Restore (hours), Pilot Light (tens of minutes), Warm Standby (minutes), Multi-Site Active-Active (near zero) – plus automated infrastructure (IaC) and runbooks.
- **Example:** Course content and student records use continuous replication to a second region (RPO of seconds/minutes) with a warm standby environment (RTO under 30 minutes); archives use daily backups.

Steps to ensure business continuity during failures

- **1. Detect:** Monitoring and alerts identify the failure or data-loss event.
- 2. Declare the incident and activate the DR plan and communication tree.
- 3. Isolate the cause (stop further deletion, revoke compromised credentials).
- 4. Restore from the latest version/backup or fail over to the standby region (DNS/traffic switch).
- 5. Validate data integrity and application health before reopening to users.
- 6. Communicate status to students and staff through status pages.
- **7. Review:** Run a post-incident analysis, update the plan, and conduct regular DR drills.

4. Government Hybrid Cloud for Sensitive and Non-Sensitive Data

Hybrid architecture with secure communication

- **On-premises/private cloud:** Hosts regulated applications and sensitive data that must stay within compliance boundaries.
- **Public cloud:** Hosts non-sensitive workloads such as public portals, analytics on anonymised data, and backup/burst capacity.
- **Connectivity:** Dedicated private links (AWS Direct Connect, Azure ExpressRoute) or site-to-site IPsec VPN between environments, terminating at a hub-and-spoke network/transit gateway.
- **Network security:** Segmentation with VPCs/subnets, firewalls, security groups, WAF, private endpoints, and API gateways; no direct internet exposure for sensitive systems.
- **Data placement:** Classification rules decide which data stays on-premises, and only approved, masked or tokenised data moves to the public cloud.

Identity management and encryption

- **Identity:** Federated single sign-on (SAML/OIDC) between on-premises Active Directory and the cloud identity provider; Multi-Factor Authentication; role-based access control with least privilege; privileged access management and periodic access reviews.
- **Encryption in transit:** TLS 1.2/1.3 for applications and IPsec/MACsec on links.
- **Encryption at rest:** AES-256 for storage, databases and backups.
- **Key management:** Customer-managed keys in an HSM-backed KMS, with key rotation, separation of duties and keys kept under government control (BYOK/HYOK) where required.
- **Logging:** Central audit logs of authentication and key usage.

Challenges in managing hybrid cloud

- Maintaining consistent security policies and compliance across two environments.
- Integration and interoperability between legacy on-premises systems and cloud services.
- Network latency, bandwidth cost and link reliability.
- Unified monitoring, visibility and incident response.
- Skill gaps, tool sprawl, data-synchronisation problems, and governance of shadow IT.
- Solutions include unified management platforms (Azure Arc, Anthos, VMware), policy-as-code, and a clear data-classification framework.

5. SaaS High Availability and Multi-Region Fault Tolerance

Multi-region deployment strategy

- Deploy the full application stack in at least two regions, each spread across multiple Availability Zones, using identical Infrastructure-as-Code templates.
- Active-active (preferred for lowest downtime) or active-passive (lower cost) modes, with a global load balancer / DNS (Route 53, Traffic Manager, Cloud Load Balancing) and health-based routing.
- Stateless application servers in each region with auto-scaling; static content served via a CDN.
- Shared nothing between regions except replicated data and global configuration, so a regional outage cannot cascade.

Data replication and failover configuration

- **Databases:** Use managed multi-region databases (Aurora Global Database, Cosmos DB, Spanner, DynamoDB Global Tables) or cross-region read replicas. Choose synchronous replication for zero data loss or asynchronous for performance with a small RPO.
- **Object storage and files:** Cross-region replication; caches and queues replicated or rebuilt per region.
- **Failover:** Health checks detect a regional failure and DNS/global load balancer automatically redirect traffic; replica databases are promoted to primary; set low DNS TTLs.
- **Conflict handling:** Use conflict-resolution rules (last-writer-wins or application logic) for active-active writes.

- **Testing:** Run regular failover drills and chaos engineering experiments.

Monitoring and alerting for proactive detection

- Collect metrics, logs and traces (CloudWatch, Azure Monitor, Prometheus/Grafana, ELK, OpenTelemetry) for latency, error rate, throughput, saturation and replication lag.
- Synthetic monitoring from multiple regions to test user journeys continuously.
- **Alerts:** Threshold and anomaly-based alerts routed to on-call tools (PagerDuty, Opsgenie, Slack) with severity levels and escalation policies.
- Dashboards, SLO/SLI tracking and error budgets, plus runbooks and automated remediation (auto-restart, auto-scale).
- Use health checks, distributed tracing and capacity forecasting to detect issues before users are affected.

SET 2 – Big Data, Security, Serverless, ETL and Machine Learning

1. Retail Near-Real-Time Analytics on Transaction Data

Cloud-based big data architecture

- **Ingestion:** Online and in-store (POS) transactions are published to a streaming platform such as Apache Kafka / Amazon Kinesis / Azure Event Hubs; bulk files arrive through batch ingestion (Data Factory, Glue, Sqoop).
- **Distributed storage:** A data lake on object storage (S3 / ADLS / GCS) using HDFS-compatible, columnar formats (Parquet/ORC) and Delta/Iceberg tables.
- **Distributed processing:** Apache Spark (EMR, Databricks, Dataproc) for batch and Spark Structured Streaming / Flink for real-time processing.
- **Serving:** Results go to a data warehouse, a low-latency store (Redis/Cassandra) for recommendations, and BI dashboards.

Integrating batch and stream processing

- **Lambda architecture:** A batch layer computes accurate views over full history, a speed layer computes real-time views, and a serving layer merges both.
- **Kappa/unified approach:** One streaming engine (Spark/Flink) handles both stream and replayed historical data from Kafka, reducing duplicate code.
- **Example:** Streaming updates inventory and recommendations within seconds; nightly batch jobs retrain models and reconcile totals.

Using data lakes and data warehouses together

- The lake stores raw, semi-structured and unstructured data cheaply (bronze), refined data (silver) and curated data (gold).
- The warehouse (Redshift, Snowflake, BigQuery, Synapse) stores modelled, structured data for fast SQL and BI.
- **Lakehouse pattern:** Open table formats let both be queried with a shared catalogue, so data scientists use the lake and business analysts use the warehouse.

Scalability and cost efficiency

- Separate storage and compute and use autoscaling clusters or serverless engines (Athena, BigQuery).
- Use Spot/preemptible instances for batch jobs; auto-terminate idle clusters.
- Partition and compress data, use columnar formats, and apply lifecycle policies to move old data to cheaper tiers.
- Cache hot data, use materialised views, and monitor spend with budgets and tagging.

2. Healthcare Cloud Migration with Strict Privacy Compliance

Secure cloud architecture

- Isolate workloads in private networks (VPC/VNet) with segmented subnets, private endpoints, firewalls and WAF; no public access to storage or databases.

- Store patient records in encrypted managed databases and medical images (DICOM) in encrypted object storage or a managed healthcare service (HealthLake, Azure Health Data Services, Cloud Healthcare API).
- Doctors access records via VPN or zero-trust access through an API gateway with single sign-on.
- Choose compliant regions and sign Business Associate Agreements (HIPAA) or equivalent data-protection agreements (GDPR).

Encryption, IAM and key management

- **Encryption:** TLS 1.2+ in transit; AES-256 at rest for databases, images and backups; field-level encryption for highly sensitive data.
- **IAM:** Role-based and attribute-based access control, least privilege, MFA, SSO with federation, short-lived credentials, and break-glass emergency access procedures.
- **Key management:** Use a KMS/HSM with customer-managed keys, automatic rotation, separate duties for key administrators and data users, and key-usage logging.

Auditing, logging and compliance monitoring

- Enable audit trails (CloudTrail, Azure Activity Log) and access logs for every record view or change.
- Send logs to a centralised, immutable SIEM (Splunk, Sentinel, Chronicle) with alerts for abnormal access.
- Use compliance tools (AWS Config, Security Hub, Azure Policy, Defender for Cloud) for continuous checks against HIPAA/GDPR controls, plus regular audits and penetration tests.

Preventing unauthorised access and breaches

- Zero-trust model, MFA, network segmentation and least privilege.
- Data loss prevention, data masking and de-identification for analytics.
- Vulnerability scanning and patching, endpoint protection, and DDoS protection.
- Anomaly detection (GuardDuty / UEBA), automatic session timeouts, immutable backups against ransomware, staff security training and an incident-response plan.

3. Online Ticket Booking – Serverless Architecture

Serverless architecture

- **Frontend:** Static web/mobile app hosted on object storage with a CDN.
- **API layer:** API Gateway exposes REST/GraphQL endpoints, handles authentication (Cognito/Azure AD B2C), throttling and caching.
- **Compute:** Functions (AWS Lambda / Azure Functions / Cloud Functions) for search, seat selection, booking and payment.
- **Data:** Managed NoSQL (DynamoDB/Cosmos DB) for seat inventory and bookings, plus a managed relational database where transactions need it; a cache for hot events.
- **Messaging:** Queues/topics (SQS, SNS, EventBridge) decouple booking, payment, email/SMS notification and reporting.

Event-driven computing and scalability

- Functions run only when triggered by events (HTTP request, queue message, database change), and the platform creates as many instances as needed within seconds.
- A sudden spike after an event announcement is absorbed automatically, with no capacity planning and pay-per-execution billing.
- Loose coupling lets each component scale independently and failures are isolated and retried.

Managed databases, API gateways and message queues

- **Managed databases:** Provide auto-scaling, replication, backup and high availability without administration; conditional writes prevent double-booking.
- **API gateways:** Single entry point providing authentication, rate limiting, caching, request validation and monitoring.
- **Message queues:** Buffer bursts of requests, smooth load on payment systems, enable retries and dead-letter queues, and guarantee orders are processed reliably.

Advantages and limitations of serverless

- **Advantages:** Automatic scaling, no server management, cost-effective for spiky workloads, fast development, built-in high availability.
- **Limitations:** Cold-start latency, execution-time and memory limits, vendor lock-in, difficult debugging and local testing, database connection limits, and unpredictable costs at very high constant load.
- **Mitigation:** Provisioned concurrency, caching, connection pooling/proxies, queues for rate control, and IaC (SAM/Serverless Framework) for portability.

4. Multinational Centralised Analytics – ETL/ELT Pipeline

Cloud-based ETL/ELT pipeline design

- **Sources:** Sales (CRM), finance (ERP), manufacturing (MES/IoT) and support (ticketing) systems.
- **Ingestion:** Connectors and change-data-capture (Fivetran, Airbyte, Data Factory, Glue, Debezium) load raw data into a landing zone/data lake.
- **ELT:** Raw data is loaded first into the cloud warehouse/lakehouse (Snowflake, BigQuery, Redshift, Databricks), then transformed using SQL/dbt/Spark into staging, integrated and curated (star-schema) layers.
- ETL is used when data must be cleansed or masked before loading.
- **Consumption:** BI tools, ML and APIs read from curated marts.

Data quality, metadata management and governance

- **Data quality:** Validation rules, schema checks, de-duplication, standardised formats and reference data, with automated tests (Great Expectations, dbt tests) and quarantine of bad records.
- **Metadata:** A data catalogue (Glue Data Catalog, Purview, Collibra, Alation) holding technical, business and operational metadata and data lineage.

- **Governance:** Data owners and stewards, classification, role-based access, masking of sensitive fields, retention rules, audit logs and compliance with GDPR and regional laws; master data management for consistent customer and product definitions.

Workflow orchestration and scheduling

- Tools such as Apache Airflow, Azure Data Factory, AWS Step Functions or Prefect define pipelines as dependency graphs (DAGs).
- They schedule jobs, handle retries, alerts, SLA monitoring, backfills and parameterised runs, and trigger downstream tasks only after upstream success.

Optimising query performance and storage cost

- Partitioning, clustering/sorting keys and columnar formats with compression.
- Materialised views, pre-aggregated tables, result caching and incremental loads.
- Right-sized, auto-suspending warehouse compute and workload isolation.
- Tiered storage with lifecycle policies, deletion of duplicate or unused data, and query monitoring to find costly queries.

5. Logistics Delay Prediction – ML and Analytics Platform

Cloud-based ML and analytics architecture

- **Ingestion:** Live GPS streams through Kafka/Kinesis/IoT Hub; weather via API connectors; historical shipments from the transport management system via batch loads.
- **Storage:** Data lake for raw and processed data, plus a feature store (SageMaker Feature Store, Feast, Vertex) for consistent training and serving features.
- **Processing:** Spark/Flink for batch and streaming feature engineering.
- **ML platform:** Managed training (SageMaker, Azure ML, Vertex AI), model registry, and real-time endpoints; results feed dashboards and dispatch applications.

Big data technologies for training and inference

- **Training:** Distributed processing (Spark MLlib, distributed XGBoost, TensorFlow/PyTorch on GPU clusters) handles large historical datasets.
- **Inference:** Low-latency online inference combines live GPS and weather features to predict delays in seconds; batch inference scores all open shipments periodically.
- Streaming joins enrich each GPS event with route, traffic and weather context.

Cloud-native AI/ML services for deployment and monitoring

- **Deployment:** Managed endpoints with autoscaling, A/B testing, canary and shadow deployments; containerised models on Kubernetes where needed.
- **Monitoring:** Track prediction latency, accuracy, data drift and concept drift (SageMaker Model Monitor, Azure ML monitoring, Vertex AI).
- **MLOps:** CI/CD pipelines, experiment tracking (MLflow), model versioning and rollback, with alerts when performance drops.

Large-scale ingestion, feature engineering and model updates

- **Ingestion:** Partitioned streaming topics, back-pressure handling, schema registry and idempotent writes.
- **Feature engineering:** Compute time-window features (average speed, delay history, weather severity, route congestion) once in the feature store and reuse for training and inference.
- **Model updates:** Scheduled and drift-triggered retraining pipelines (continuous training), automatic validation against the current model, and gradual rollout.
- **Cost control:** Spot instances for training and autoscaling inference endpoints.

SET 3 – Migration, IoT, BI, Monitoring and Data Lifecycle

1. Manufacturing Company Cloud Migration

Cloud migration strategy

- **Phase 1 – Assess:** Discover applications, dependencies, performance baselines, licensing and compliance needs (Azure Migrate, AWS Application Discovery).
- **Phase 2 – Plan:** Classify each application using the 6 Rs, define the target landing zone (network, identity, security, governance), and agree a migration wave plan.
- **Phase 3 – Migrate:** Start with low-risk applications, then move critical ones in waves using continuous replication.
- **Phase 4 – Optimise:** Right-size, apply cost controls, modernise and decommission legacy systems.

Applying the migration approaches (6 Rs)

- **Rehost (lift-and-shift):** Move legacy servers as-is to cloud VMs for speed.
- **Replatform:** Make small changes, such as moving a self-managed database to a managed database service.
- **Refactor/Re-architect:** Redesign critical applications into microservices/containers to gain scalability and cloud-native benefits.
- **Repurchase:** Replace legacy software with SaaS (e.g., CRM or HR system).
- **Retire:** Switch off unused or duplicate applications.
- **Retain:** Keep certain systems on-premises (e.g., shop-floor control or systems with latency/regulatory constraints) in a hybrid setup.

Validating performance after migration

- Compare against pre-migration baselines for response time, throughput, CPU, memory, I/O and latency.
- Run functional, integration, load and stress tests and user acceptance testing.
- Use monitoring and APM tools (CloudWatch, Azure Monitor, Datadog) plus data-integrity checks and checksum comparisons.
- Run in parallel for a trial period before final cut-over; tune instance sizes and network settings.

Reducing migration risk and downtime

- Use continuous data replication (DMS, Azure Site Recovery) and a short final cut-over window.
- Use pilot migrations, blue-green or phased cut-over and a tested rollback plan.
- Take full backups before migration; schedule migrations in low-activity windows.
- Secure the landing zone, provide staff training, and communicate clearly with stakeholders.

2. Smart City IoT Real-Time Analytics

Cloud architecture for IoT collection and analytics

- **Edge/device layer:** Sensors connect through gateways using MQTT/CoAP; edge devices filter and pre-process data.
- **Ingestion:** IoT hub/core (AWS IoT Core, Azure IoT Hub) provides device registry, authentication and a message broker.
- **Processing:** Stream-processing engines for real-time rules and analytics; batch jobs for trends.
- **Storage:** Time-series database for recent data, data lake for history, and a warehouse for reporting.
- **Presentation:** Dashboards, alerts and APIs for traffic control, emergency services and city planners.

Integrating message brokers, stream processing and storage

- Sensors publish to a broker (MQTT broker / Kafka / Pub-Sub), which buffers and decouples producers and consumers.
- Stream processors (Spark Streaming, Flink, Kinesis Analytics) consume messages, clean, aggregate in time windows and detect anomalies, such as an air-quality threshold.
- Outputs are written simultaneously to hot storage (time-series/NoSQL), the data lake (raw archive) and alerting services.

Handling high-volume, high-velocity data

- Partitioned topics, horizontal scaling of consumers and back-pressure handling.
- Edge computing and data aggregation/down-sampling to cut volume.
- Compression, efficient binary formats (Avro/Protobuf), in-memory processing and tiered storage.

Scalability, reliability and security

- **Scalability:** Autoscaling brokers and processors, serverless components, and multi-AZ deployment.
- **Reliability:** Replicated message queues, retries, dead-letter queues, redundant gateways and offline buffering on devices.
- **Security:** Unique device certificates, mutual TLS, secure boot and firmware updates, device identity and policies, encryption at rest, network segmentation, and monitoring for compromised devices.

3. Global E-Commerce Business Intelligence Dashboards

Cloud-based BI architecture

- **Sources:** Order, web-clickstream, CRM, inventory and marketing systems.
- **Ingestion:** Streaming (Kafka/Kinesis) for real-time events and batch ETL/ELT for historical data.
- **Storage and modelling:** Data lake plus cloud data warehouse (Snowflake, BigQuery, Redshift) with star-schema marts for sales, customers and regions.

- **Semantic layer:** Defines consistent metrics (revenue, conversion, churn).
- **Visualisation:** Tableau, Power BI, Looker or QuickSight dashboards for executives, with real-time and historical views.

Integration of visualisation tools with cloud data warehouses

- Native connectors (JDBC/ODBC, built-in drivers) provide either live/direct query for real-time data or scheduled extracts/in-memory import for speed.
- Single sign-on and row-level security pass user identity to the warehouse.
- Embedded analytics and APIs allow dashboards inside internal applications.

Optimising dashboard performance with large datasets

- Pre-aggregate data and use materialised views and summary tables.
- Partitioning, clustering and indexing on commonly filtered columns (date, region).
- Query result caching, extracts, incremental refresh and limiting visuals per dashboard.
- Use filters, drill-down instead of loading all detail, and scale warehouse compute during peak hours.

Security measures for reports and sensitive data

- Role-based access control and row/column-level security by region or department.
- Encryption in transit and at rest, MFA/SSO, and data masking for sensitive fields.
- Audit logs of report access and exports, restricted download/sharing, watermarking, network controls (private endpoints/VPN), and compliance with GDPR/CCPA.

4. Online Gaming – Monitoring and Performance Optimisation

Cloud monitoring and performance optimisation solution

- Deploy agents and exporters on game servers, databases and load balancers to collect metrics, logs and traces into a centralised platform (CloudWatch, Azure Monitor, Prometheus/Grafana, Datadog, ELK).
- Build dashboards for latency, concurrent players, error rate and resource usage.
- Configure alerts, automated remediation and autoscaling policies tied to these metrics.

Using metrics, logs and tracing to find bottlenecks

- **Metrics:** CPU, memory, network, disk I/O, request latency, queue depth and player counts show when and where resources saturate.
- **Logs:** Show errors, timeouts, slow queries and failed matchmaking events.
- Distributed tracing (OpenTelemetry, X-Ray, Jaeger): Follows a request through matchmaking, game server, database and cache to pinpoint the slow component.
- Correlating the three explains, for example, that peak-hour latency comes from database connection limits.

Resource optimisation and capacity planning

- Autoscaling groups with scheduled scaling for known peaks and predictive scaling from historical trends.

- Right-sizing instances, caching (Redis), database read replicas and sharding, and CDN for game assets.
- Load testing and forecasting future growth; place servers in regions near players to reduce latency.

Improving availability while controlling cost

- Multi-AZ deployment, health checks and automatic failover.
- Reserved/Savings Plans for baseline capacity, Spot instances for non-critical jobs, and scale-in during off-peak hours.
- Budgets, tagging and cost alerts; remove idle resources; use managed services to cut operations effort.

5. Financial Institution – Data Lifecycle Management

Cloud-based data lifecycle strategy

- **Define stages:** Create/ingest → Active use → Infrequent access → Archive → Legal hold/retention → Secure deletion.
- Hot tier (recent transactions) on standard storage or warehouse; warm tier on infrequent-access storage; cold tier on archive storage (S3 Glacier, Azure Archive, Coldline).
- Automate transitions with lifecycle policies and keep metadata in a catalogue so archived data can still be found.

Data classification, retention policies and archival storage

- **Classification:** Label data as public, internal, confidential or restricted (PII, financial) using automated discovery tools (Macie, Purview).
- **Retention:** Set periods based on regulation (e.g., 5–10 years for transaction records), apply legal holds and WORM/object lock so records cannot be altered.
- **Archival:** Compress and encrypt data, store in low-cost archive tiers with defined retrieval times, and automatically delete data when retention expires.

Governance, auditing and regulatory compliance

- Governance assigns ownership, policies and stewardship so data is trustworthy and used appropriately.
- **Auditing:** Immutable logs of access and changes prove compliance and support investigations.
- Regulations (PCI-DSS, GDPR, SOX, Basel/RBI or local rules) demand retention, privacy, residency and reporting; non-compliance risks fines and reputation damage.
- Use compliance monitoring tools and periodic audits.

Balancing accessibility, security and cost

- **Tier data by access frequency:** fast storage for hot data, cheaper tiers for cold data, with clear retrieval SLAs.
- Apply encryption, KMS, least-privilege access and MFA at every tier.
- Use intelligent tiering, compression, columnar formats and de-duplication to cut cost.

- Monitor storage usage and costs regularly, and test archive restores to confirm records remain accessible when needed.